AI Voice Sales & Support Agent

Project Report

A Real-Time, Hybrid AI Voice Booking Agent for Inbound Calls

Course "AI5301-Advanced Artificial Intelligence**"**

Submitted To: "Dr. Sajid Mahmood"

Submitted By: "Arham Shahzad**"**

Student ID: "S2026436030**"**

Submitted By: "Haseeb Ahmad**"**

Student ID: "S2026436009**"**

Submission Date: 25 June 2026

Departments of Artificial Intelligence (AI)

School of Science and Technology (UMT SST)

1. Introduction

The AI Voice Sales & Support Agent is a real-time, telephone-based conversational AI system designed to handle inbound customer calls without human intervention. Rather than routing callers through a traditional IVR menu or a human receptionist, the system answers the call, converses naturally using synthesized speech, and walks the caller through a structured booking flow — capturing service requirements, contact details, and scheduling preferences — before politely ending the call. The current implementation is configured for a home-services use case (HBAG Home Services), handling categories such as plumbing, electrical, and AC repair requests, but the architecture is general enough to be repointed at other booking or support scripts.

This report documents the system's purpose, architecture, the backend and frontend implementations, the conversational design, and an assessment of its current capabilities and limitations.

2. Project Objectives

- Provide 24/7 automated phone coverage so that no inbound call goes unanswered.
- Conduct a natural, low-latency, two-way voice conversation rather than a rigid menu-based IVR.
- Reliably extract and store structured booking data (service type, issue, name, date, address) from unstructured speech.
- Support real-time monitoring of live calls via a transcript dashboard.
- Keep voice latency low enough to feel conversational, including the ability for the caller to interrupt (barge-in).

3. System Architecture

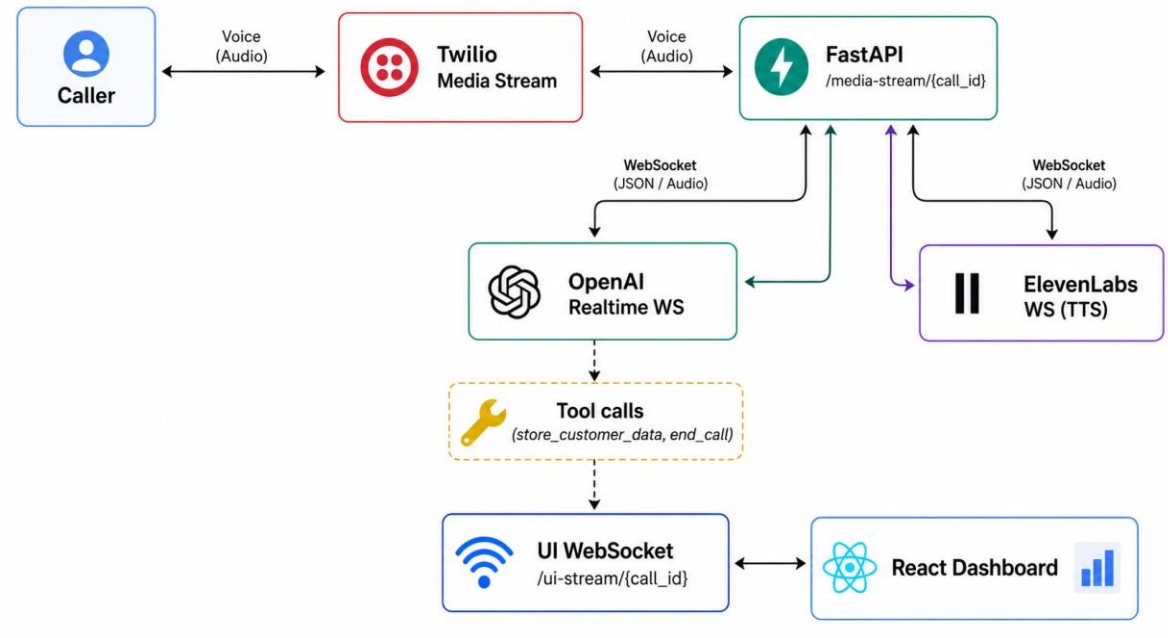
The system is composed of four cooperating layers: telephony, conversational reasoning, voice synthesis, and a backend orchestration layer that ties them together over WebSockets. A separate React frontend provides a way to place a simulated call from the browser for testing, without needing a real phone line.

3.1 High-Level Call Flow

1. A call arrives at Twilio, which POSTs to the backend's /incoming-call webhook (or, for testing, the frontend calls /test-call directly).
2. The backend creates a unique call_id, registers the call in an in-memory active_calls table, and returns TwiML instructing Twilio to open a Media Stream WebSocket back to the backend.
3. The backend opens a second WebSocket connection to the OpenAI Realtime API, configured with the booking system prompt, audio formats (G.711 μ-law), server-side voice activity detection, and the available function tools.
4. Caller audio is forwarded, in real time, from the Twilio media socket into the OpenAI Realtime socket. OpenAI transcribes, reasons over, and responds to the caller's speech.
5. Instead of using OpenAI's native audio output, the backend intercepts the assistant's streamed text transcript and forwards it to ElevenLabs for higher-fidelity speech synthesis (a 'hybrid audio' approach).
6. Synthesized audio is streamed back to Twilio (and to the browser test client) in real time, and a parallel UI WebSocket broadcasts transcript and status events to any connected dashboard.

7. When the booking flow completes, the model invokes an end_call function, the backend announces the call as ended, waits briefly for final audio to play out, and closes the connection.

3.2 Component Diagram



3.3 Technology Stack

Layer	Technology	Role
Telephony	Twilio Voice	Receives the phone call and streams raw call audio to the backend over a WebSocket
Backend API	FastAPI + Uvicorn	HTTP webhooks, WebSocket endpoints, async orchestration of the whole call
Conversational AI	OpenAI Realtime API (gpt-realtime-1.5)	Speech understanding, dialogue reasoning, transcript generation, function calling
Voice Synthesis	ElevenLabs (eleven_turbo_v2_5)	Converts the agent's streamed text into natural-sounding μ-law audio
Frontend	React 19 + Vite	Browser-based test client: places a simulated call,

		captures mic audio, plays agent audio, shows live transcript
Networking	Python websockets, Axios	Async WebSocket client/server and HTTP requests between layers

4. Backend Implementation

The backend is a single FastAPI application (main.py) supported by four modules: config.py, tools.py, hybrid_audio.py, and elevenlabs_service.py.

4.1 main.py — Call Orchestration

This module defines all HTTP and WebSocket endpoints and contains the core call loop:

- POST /incoming-call — Twilio webhook entry point; registers the call and returns TwiML pointing Twilio's Media Stream at the backend's WebSocket.
- POST /test-call — a lightweight endpoint used by the React dashboard to simulate a call without Twilio, returning the WebSocket URLs the frontend should connect to.
- WS /ui-stream/{call_id} — a broadcast channel that pushes transcript lines, speaking-state, and call-status events to any connected dashboard.
- WS /media-stream/{call_id} — the heart of the system. On connect, it initializes the HybridAudioService, opens a connection to the OpenAI Realtime API, configures the session (instructions, audio formats, VAD thresholds, available tools), and triggers an initial greeting.

Three concurrent asyncio tasks drive each call:

- receive_from_client() — reads incoming Twilio/browser audio frames and forwards them to OpenAI, pausing while the agent is speaking.
- process_openai() — consumes events from the OpenAI Realtime socket: speech transcripts, audio-transcript deltas, user speech-started events (used to interrupt/clear playback), and function-call requests.
- stream_elevenlabs_to_client() — drains synthesized audio from the hybrid audio service and streams it back to the caller as Twilio media events.

Function calling is handled generically: when OpenAI requests a tool (store_customer_data or end_call), the backend looks it up in a TOOLS registry, executes it, broadcasts the result to the dashboard, and feeds the output back into the OpenAI conversation so the model can continue naturally.

4.2 config.py — Configuration & Conversation Script

Centralizes environment-driven configuration (API keys for OpenAI, ElevenLabs, Twilio, and MongoDB; the public BASE_URL used to build WebSocket URLs) and defines the BOOKING_PROMPT — a strict, step-by-step system prompt that constrains the model to a fixed five-question intake flow: service type, issue details, customer name, preferred date, and address, followed by a confirmation read-back and call termination. This deterministic prompt design is what keeps the conversation focused and prevents the model from wandering off-script or hallucinating unrelated knowledge-base answers.

4.3 tools.py — Function Tools

Defines the two functions exposed to the OpenAI Realtime model via its tool-calling interface:

- store_customer_data(call_id, data_type, value) — persists a single piece of confirmed booking information in an in-memory VERIFIED_DATA store, keyed by call.
- end_call(call_id) — flags the call for termination once the booking summary has been confirmed.

Each tool is described with a JSON schema (TOOLS_CONFIG) that OpenAI uses to decide when and how to invoke it during the conversation.

4.4 hybrid_audio.py & elevenlabs_service.py — Low-Latency Speech Synthesis

Rather than relying on OpenAI's built-in voice output, the system routes the assistant's text transcript through ElevenLabs for synthesis — a 'hybrid' design intended to combine OpenAI's reasoning quality with ElevenLabs' voice quality. Key engineering details:

- Sentence-aware buffering: elevenlabs_service.py accumulates streamed text and flushes it to ElevenLabs at sentence boundaries, so synthesis can start before the model has finished the entire reply.
- Connection pre-warming: hybrid_audio.py opens a fresh ElevenLabs WebSocket connection as soon as the caller starts speaking (in parallel with the model 'thinking'), so that by the time a response is ready, the TTS connection has effectively zero added latency.
- Barge-in handling: if the caller starts talking while the agent is still speaking, the backend immediately sends a 'clear' event to flush buffered audio and stops the current ElevenLabs stream, allowing natural interruption.
- Streaming playback: audio is yielded chunk-by-chunk through an async generator and forwarded to Twilio as it arrives, rather than waiting for the full utterance to be synthesized.

5. Frontend Implementation

The frontend is a single-page React application (App.jsx) built with Vite. Its purpose is to let a developer or stakeholder place a test call directly from a browser, without needing a real phone number, while watching the conversation unfold as a live chat-style transcript.

5.1 Key Responsibilities

- Initiates a test call by POSTing to /test-call and opening two WebSockets: one for the live transcript (ui_ws) and one that mimics a Twilio Media Stream (media_ws).
- Captures microphone audio via getUserMedia, downsamples it from 48kHz to 8kHz, encodes it to G.711 μ-law, and streams it to the backend as base64-encoded media frames — replicating exactly what Twilio would send.
- Decodes incoming μ-law audio chunks from the agent, converts them back to PCM, and plays them through the Web Audio API in real time.
- Renders a live, auto-scrolling transcript with distinct styling for the caller and the agent, plus a status indicator (idle / connecting / active / ended).

Because the frontend implements the same μ-law encode/decode and Twilio-style message protocol that a real phone call would use, it is an effective stand-in for end-to-end testing without needing a live Twilio number during development.

5.2 Frontend Stack

Dependency	Purpose
React 19 / Vite	UI rendering and fast dev/build tooling
Axios	HTTP requests to the backend's /test-call endpoint
Web Audio API	Microphone capture, μ-law encode/decode, and audio playback (no external audio library)

6. Conversational Design

The booking flow is intentionally rigid by design: a fixed, single-purpose system prompt instructs the model to ask exactly one question at a time, confirm each answer, store it via a function call, and never skip a step. This trades some conversational flexibility for reliability — important in a voice channel where ambiguous, multi-part questions are easy to mishear. The flow is:

8. Greeting
9. Ask service type → confirm → store
10. Ask issue details → acknowledge → store
11. Ask customer name → confirm → store
12. Ask preferred date → confirm → store
13. Ask home address → confirm → store
14. Read back a full summary, ask for confirmation, then end the call

7. Current Capabilities & Limitations

7.1 Strengths

- Low-latency, naturally interruptible voice conversation via server-side VAD and connection pre-warming.
- Clear separation of concerns: telephony, reasoning, and synthesis are independently swappable components.
- A deterministic conversational script reduces the risk of the agent going off-topic mid-call.
- Real-time observability through the UI WebSocket broadcast, useful for QA and live monitoring.
- A working browser-based test harness that avoids dependency on a live Twilio trunk during development.

7.2 Current Limitations

- Call and booking data are held in process memory (active_calls, VERIFIED_DATA) and are lost on restart; although a MongoDB URI is read from configuration, no persistence layer is yet wired into the call flow.
- CORS is currently fully open (allow_origins="*"), which should be restricted before production deployment.
- The booking script is hard-coded for a single business (HBAG Home Services) rather than being configurable per tenant.
- There is no authentication on the dashboard WebSocket or the test-call endpoint.
- Error handling around dropped WebSocket connections (Twilio, OpenAI, ElevenLabs) is present but minimal — failures are logged rather than retried.

8. Conclusion & Next Steps

The AI Voice Sales & Support Agent demonstrates a working, low-latency, end-to-end voice AI pipeline that can answer a phone call, converse naturally, capture structured booking data, and hand off a clean summary — all without human involvement. The architecture's hybrid use of OpenAI for reasoning and ElevenLabs for synthesis, combined with careful latency engineering (sentence-level streaming, connection pre-warming, barge-in support), results in a noticeably more responsive experience than typical IVR or chatbot-style systems.

Recommended next steps to move toward production readiness:

- Wire the existing MongoDB configuration into tools.py so booking data and transcripts persist beyond a single process lifetime.
- Restrict CORS and add authentication to the dashboard and test-call endpoints.
- Externalize the booking script/prompt so multiple business configurations can be served from the same backend.
- Add automated tests around the WebSocket call flow and the μ-law audio conversion logic.
- Add call analytics (duration, completion rate, drop-off point) to the dashboard for ongoing monitoring.